

MUAMMOGA ASOSLANGAN O'QITISH (PBL)

Python → Django → Django REST

- 1) Python nima? Xususiyatlari va boshqa tillarga nisbatan yutuqlari
- 2) Muammo: veb kerak → Django
- 3) Muammo: telefon + ish stoli + sayt parallel → DRF

Qoida: avval MUAMMO, keyin YECHIM.

v8 · rasmlar aniq ko'rinadi · o'zbekcha atamalar (inglizchasi qavsda)

Python nima?

Python — umumiy maqsadli (general-purpose), yuqori darajali dasturlash tili.

Kod yoziladi, keyin talqinchi (interpreter) orqali bajariladi. Maqsad: odam o'qishi oson bo'lgan dastur yozish.

Yuqori darajali

Past darajadagi xotira buyruqlari bilan emas — masala va mantiq bilan ishlaysiz.

Talqin qilinadi (interpreted)

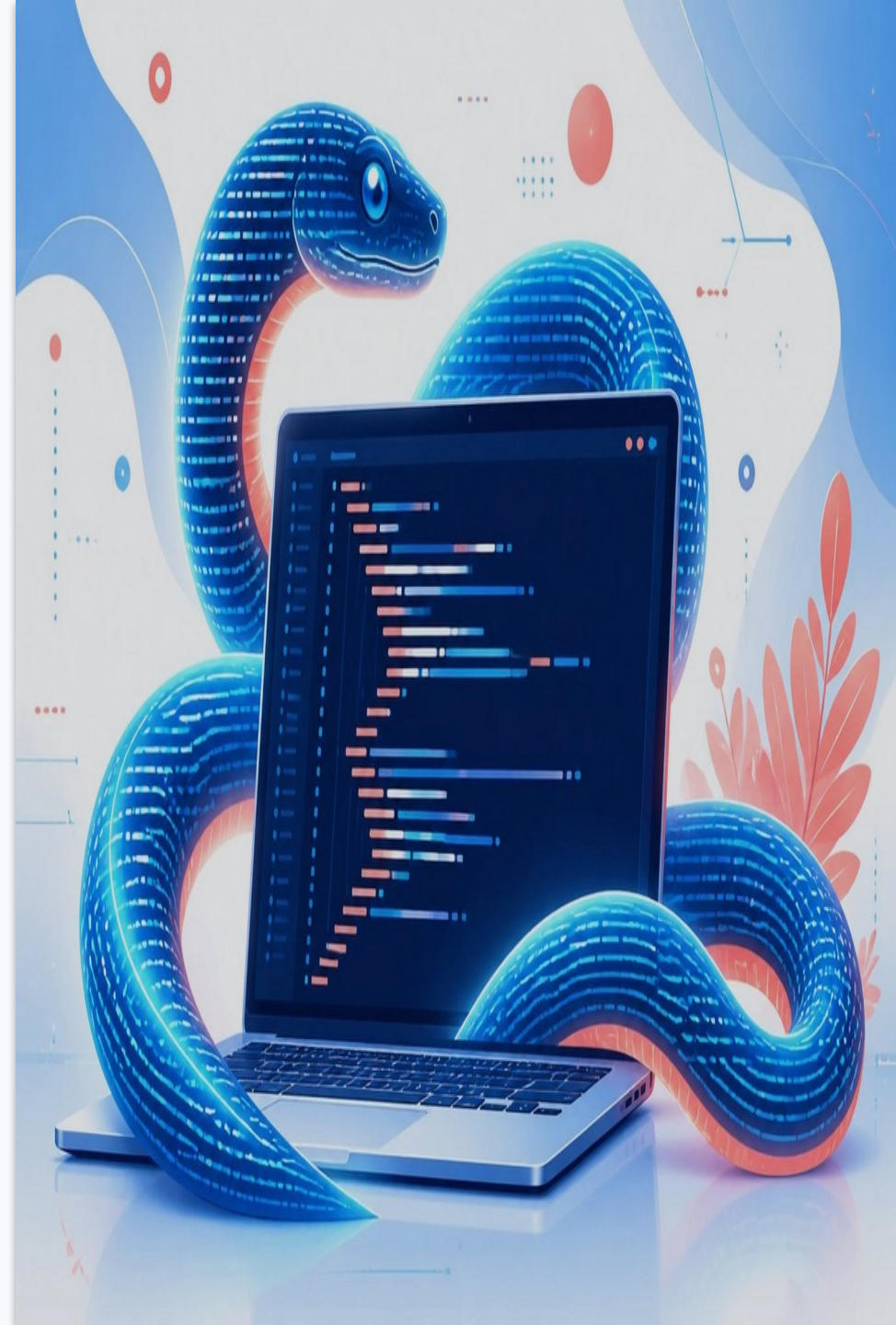
Ko'pincha alohida exe majburiy emas. .py faylni talqinchi ishga tushiradi.

Ko'p uslubli (multi-paradigm)

Oddiy tartib, obyektli (OOP), funksiyaviy uslub — bir tilda birga.

Ochiq kodli (open source)

Erkin, katta jamiyat. Windows / macOS / Linux da bir xil fikrlash.



Ichida nima bor — oson, lekin teran



Sintaksis (syntax)

Bo'sh joy bilan blok (indentatsiya) majburiy. Stil bir xil bo'ladi, o'qish osonlashadi.

O'zgaruvchan tip (dynamic typing)

Tipni majburan yozmaysiz. Qiymatga qarab aniqlanadi. Katta loyihada tip ko'rsatmasi (type hint) ixtiyoriy.

Ichki boylik (batteries included)

Standart kutubxona boy: fayl, JSON, tarmoq so'rovi, vaqt, qidiruv andozasi...

Keng ekotizim

Tashqi ombor (PyPI) orqali minglab paket: Django, FastAPI, Pandas, NumPy...

Tez ishlab chiqish

Kam ortiqcha shablon-kod (boilerplate) → prototip tez chiqadi.

Ko'chma (portable)

Bir xil kod turli tizimlarda ishlashi oson. Izolyatsiya muhiti (virtual environment) bilan boshqariladi.

Boshqa tillarga nisbatan yutuqli tomonlari

Bu “boshqa tillar yomon” degani emas — Python qayerda yutadi, shu yerda aniq aytamiz.

Java / C# ga nisbatan

Kamroq ortiqcha shablon-kod. Tip va klass e'lonlari kamroq. O'rganish egri chizig'i yumshoqroq. Veb va sun'iy intellekt prototipida tezroq.

C / C++ ga nisbatan

Xotirani qo'lda boshqarmaysiz. Ko'rsatkich (pointer) xatolari kam. Sof tezlik pastroq bo'lishi mumkin — ko'p vazifa uchun yetarli.

JavaScript ga nisbatan

JS asosan brauzer + Node. Python esa sun'iy intellekt, ma'lumot tahlili, avtomatlashtirish, veb orqa qismida kengroq tanlov.

Go / Rust ga nisbatan

Go/Rust tizim tezligi uchun kuchli. Python o'qiluvchanlik, ekotizim va tez ishlab chiqishda yutadi.

Bir xil vazifa: juft sonlarni olish

Yutuq shu yerda: kam qator, aniq ma'no, o'qish oson.

Java (uzunroq)

```
List<Integer> nums = Arrays.asList(1,2,3,4,5);
List<Integer> evens = new ArrayList<>();
for (Integer n : nums) {
    if (n % 2 == 0) {
        evens.add(n);
    }
}
System.out.println(evens);
```

Python (qisqa + aniq)

```
nums = [1, 2, 3, 4, 5]
evens = [n for n in nums if n % 2 == 0]
print(evens) # [2, 4]
```

```
# Xulosa:
# bir xil natija
# kamroq kod
# o'qish osonroq
```

Tipni kim aniqlaydi?

Ko'p tillarda

O'zgaruvchi ochilganda tip yoziladi: butun son (int), matn (String)...
Tekshiruv oldindan bo'ladi. Aniqlik yuqori, yozish uzunroq.

Python da

Tip majburiy yozilmaydi — qiymatga qarab aniqlanadi.
Tez yozish. Katta loyihada tip ko'rsatmasi (type hint) qo'shish mumkin.

Java

```
int yosh = 18;  
String ism = "Ali";  
double narx = 12.5;  
boolean aktiv = true;
```

Python

```
yosh = 18  
ism = "Ali"  
narx = 12.5  
aktiv = True  
# tipni Python o'zi biladi
```

Ro'yxat, lug'at, funksiya — kundalik vositalar

Ma'lumot tuzilmalari

```
talaba = {  
    "ism": "Ali",  
    "ball": 95,  
}  
print(talaba["ism"])  
  
fanlar = ["Python", "Django"]  
fanlar.append("REST")  
# lug'at (dict) = kalit→qiymat  
# ro'yxat (list) = tartibli to'plam
```

Funksiya (function)

```
def yigindi(a, b):  
    return a + b  
  
print(yigindi(2, 3)) # 5  
  
def salom(ism="mehmon"):  
    return f"Salom, {ism}"  
  
# Funksiya = takrorlanadigan  
# mantiqni bitta joyga yig'ish
```


Bitta til — bir necha yoʻnalish



Veb orqa qismi (backend)

Django, Flask, FastAPI.

Sayt va dasturiy interfeys (API).

Maʼlumot tahlili (data)

Pandas, NumPy.

Jadvallar ustida kuchli tahlil.

Sunʼiy intellekt (AI / ML)

PyTorch, TensorFlow.

Model oʻqitish va bashorat.

Avtomatlashtirish

Fayl, jadval, pochta, saytdan maʼlumot olish (scraping).

Ilmiy hisob

SciPy, Matplotlib.

Grafik va matematik modellar.

Skript / DevOps

Server skriptlari, yordamchi vositalar, tarmoq soʻrovlari.

Python o'zi — lekin veb kerak

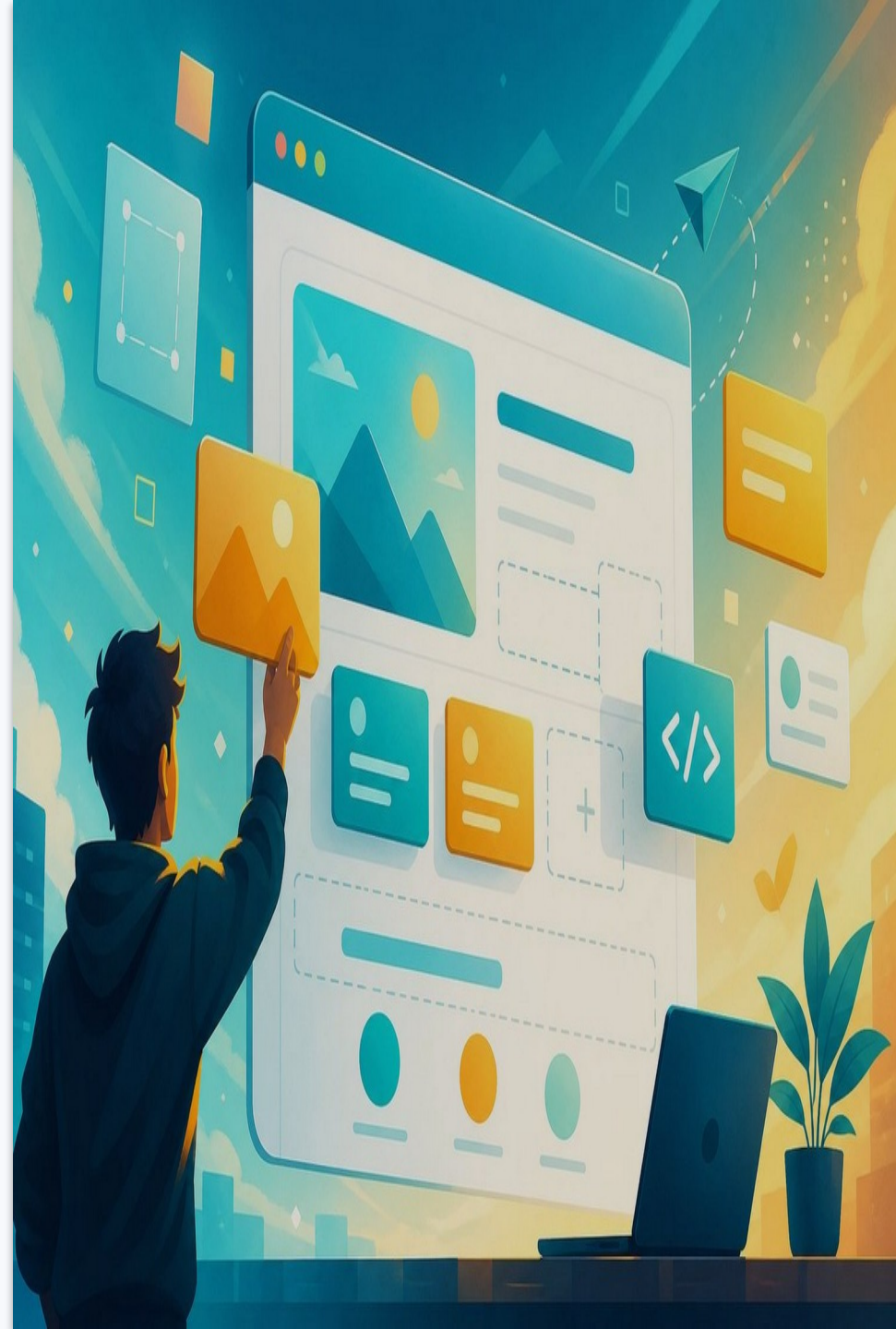
MUAMMO

Python da skript yozishni o'rgandik. Lekin real mahsulot ko'pincha brauzerda: sayt, kabinet, boshqaruv paneli, forma...

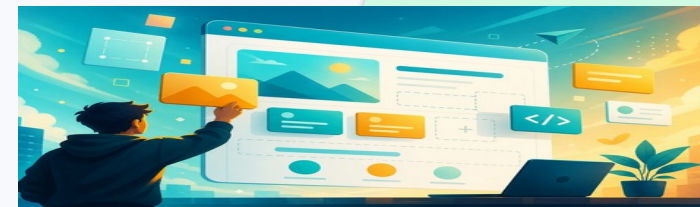
Savol: faqat print() bilan to'liq veb-tizim bo'ladimi?

Manzil yo'naltirish (URL routing), ma'lumotlar bazasi, kirish (login), xavfsizlik, sahifa andozasi — bularni noldan yig'ish og'ir.

Shuning uchun boshlang'ich veb uchun dasturiy asos (framework) tanlaymiz.



Boshlang'ich veb uchun asosiy tavsiya



**Django — Python dagi to'liq veb dasturiy asosi (full-stack framework).
Ichida tayyor: kirish tizimi, boshqaruv paneli, ORM, sahifa andozasi, xavfsizlik.**

Nima uchun Django?

Boshlang'ich uchun yo'l aniq: loyiha/ilova (Project/App), manzil (URL), boshqaruvchi (View), model, andoza (Template).

Nima beradi?

Kirish/ro'yxat, admin panel, baza bilan ORM, HTML sahifalar, himoya vositalari (CSRF va boshqalar).

Qanday o'rganamiz?

Avval so'rov qanday yurishini tushunamiz. Keyin kichik sayt. Keyin dasturiy interfeys (API).

Bitta so'rovning (request) ketma-ketligi

1. So'rov (request)

Brauzer manzil ochadi
yoki forma yuboradi

2. Manzil (URL)

Django to'g'ri yo'lni
topadi (urls.py)

3. Boshqaruvchi (View)

Mantiq ishlaydi:
tekshiruv, hisob

4. Model (Model)

Kerak bo'lsa bazadan
o'qiydi / yozadi

5. Andoza (Template)

HTML sahifa yasaydi
(klassik usul)

6. Javob (response)

Natija brauzerga
qaytadi

Uchta rol — eslab qolish oson

Model (Model)

Ma'lumot modeli.

Masalan: User, Product, Post.

ORM orqali SQL kamroq yoziladi — Python klasslari bilan baza boshqariladi.

Andoza (Template)

Ko'rinish (HTML).

Foydalanuvchi brauzerda shuni ko'radi.

Boshqaruvchidan kelgan ma'lumot sahifaga joylanadi.

Boshqaruvchi (View)

Mantiq markazi.

So'rov keldi → qaror:
qaysi model? qaysi andoza?

Javobni tayyorlaydi.

Faqat sayt yetarli emas — real hayot

MUAMMO

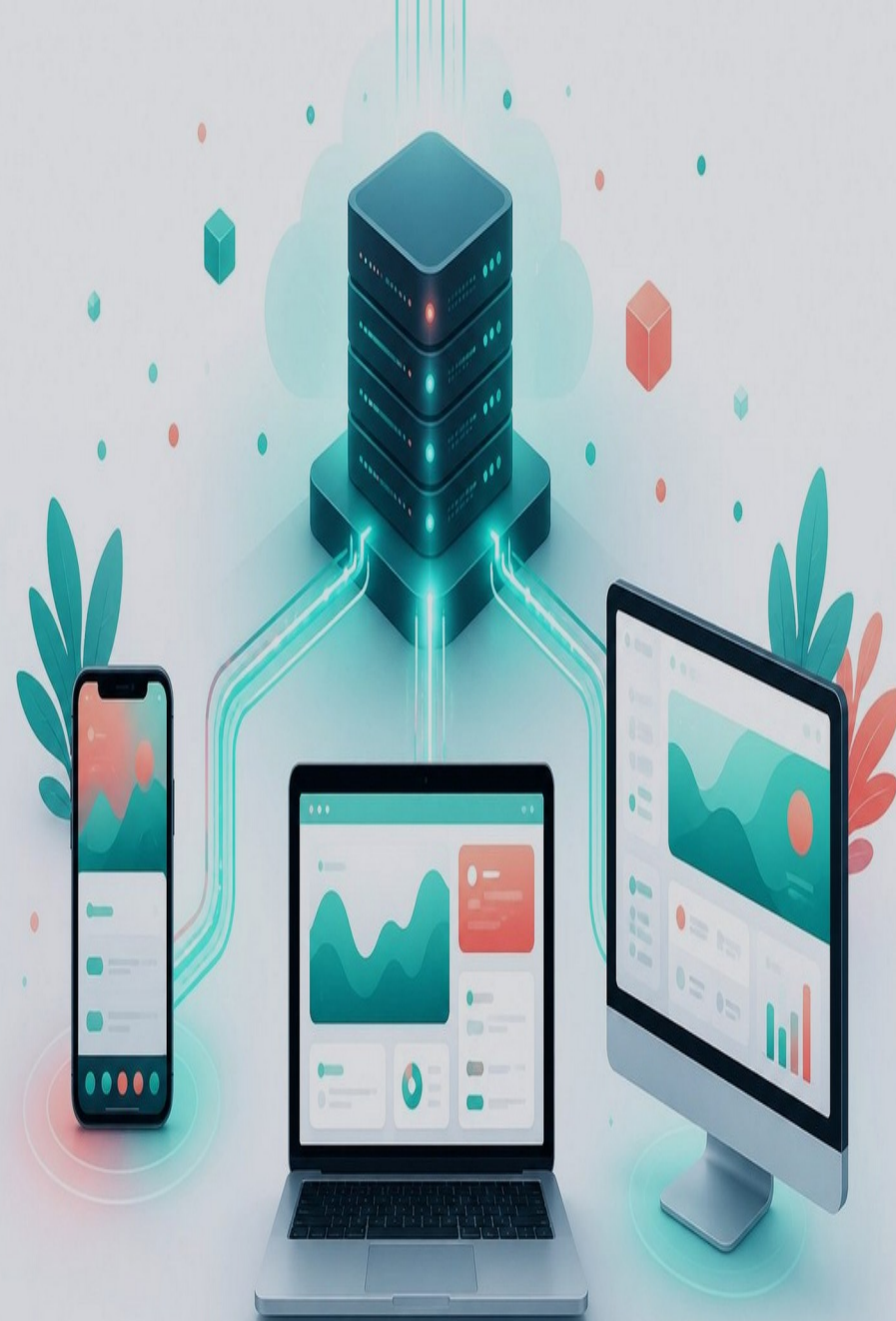
Biz yozgan kod faqat kompyuter brauzerida emas.

Telefon ilova + ish stoli dasturi (desktop) + veb — parallel ishlashi kerak.

Klassik Django ko'pincha HTML qaytaradi.

Mobil/ish stoli ilova esa HTML emas — ma'lumot (odatda JSON) so'raydi.

Xulosa-muammo: faqat andoza (Template) bilan bitta orqa qismni uchala platformaga to'g'ri ulash qiyin. Kuchliroq tushuncha kerak: dasturiy interfeys (API).



Uchta mijoz — bitta ma'lumot

Telefon (mobile)

Mobil ilova.

HTML emas — dasturiy interfeys (API) kerak.

Ish stoli (desktop)

Kompyuter dasturi.

Ham ma'lumot so'raydi.

Veb (web)

Brauzer / yagona sahifa ilova (SPA).

HTML yoki API.

Savol: mahsulot/foydalanuvchi ma'lumotini uchallasiga qanday beramiz?

Agar faqat HTML bersak — telefon va ish stoli “tushunmaydi”.

Yechim sifatida Django REST Framework (DRF) tavsiya qilinadi.

Bir orqa qism — ko'p platforma



DRF — Django ustida dasturiy interfeys (API) qatlami. Ma'lumotni JSON qilib beradi. Telefon, ish stoli, veb — parallel ulanadi. Django o'rnini bosmaydi — uni kengaytiradi.

Nima qiladi?

HTML o'rniga JSON.

Masalan: `/api/products/`

Serializator, ViewSet, yo'naltirgich (router), autentifikatsiya — API uchun asosiy vositalar.

Nima uchun kerak?

Bir marta yozilgan biznes-mantiq barcha mijozlarga xizmat qiladi.

Real hayotdagi "parallel ishlash" shu yerda yechiladi.

O'rganish tartibi

- 1) Python asoslari
- 2) Django (sayt)
- 3) DRF (API)

Avval muammo, keyin yechim — shu tartibda.

XULOSA

Bugungi darsdan olib ketiladigan asoslar

1

Python

Umumiy maqsadli til: o'qilishi oson, tipni o'zi aniqlaydi, ekotizimi boy.

2

Django

Boshlang'ich veb uchun to'liq dasturiy asos (framework) — asosiy tavsiya.

3

Muammo

Faqat sayt yetarli emas: telefon + ish stoli + veb parallel kerak.

4

DRF

Yechim: bitta dasturiy interfeys (API) orqali barcha platformalar ulanadi.